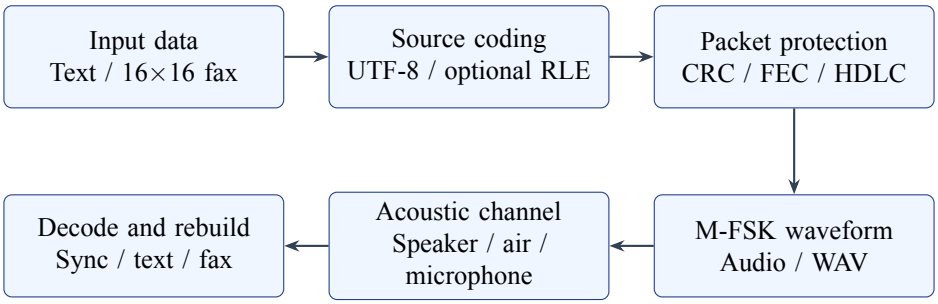
Technical Project Report

SonicDrop

A Software-Defined Acoustic Data Transceiver

Reliable text and binary-image transfer over a sound channel



Author: Reza Ahmadizadeh
Project: Acoustic Modem (SonicDrop)
Implementation: Python, Django, NumPy, browser audio APIs
Report date: November 9, 2026

Abstract

SonicDrop is a software-defined modem and web application for sending short text messages and small binary images through sound. The sender converts application data into packets, adds a compact header and CRC-8, optionally applies Run-Length Encoding (RLE) and Hamming(7,4) forward error correction, frames the bit stream using HDLC-style flags and bit stuffing, and synthesizes a mono M-FSK waveform. The receiver accepts a WAV recording or microphone audio, detects candidate tones, searches several symbol-timing offsets, checks frames, repairs correctable bit errors, and reconstructs packet content. The interface also exposes simulated channel noise, signal metrics, frame inspection, and a browser-coordinated acoustic ACK/NACK demonstration. This report describes the implemented pipeline and its assumptions, distinguishes nominal calculations from measured results, and identifies important prototype limitations—in particular, the custom passkey transform is not production-grade cryptography and the browser ARQ demonstration is not a complete transport protocol.

Contents

1 Introduction **3**

1.1 Project goals . . . 3

1.2 Scope and terminology . . . 3

2 System overview **3**

2.1 Repository components . . . 3

3 End-to-end operation **3**

3.1 Transmission endpoint . . . 4

3.2 Reception endpoint . . . 4

3.3 Live microphone and browser coordination . . . 4

4 Source coding and packet format **5**

4.1 Text and image representation . . . 5

4.2 Run-Length Encoding . . . 5

4.3 Packet header . . . 5

4.4 Optional passkey transform and security limits . . . 6

5 Data-link framing and error control **6**

5.1 CRC-8 . . . 6

5.2 Hamming(7,4) forward error correction . . . 6

5.3 HDLC-style frame boundaries . . . 6

5.4 Packet delivery and retransmission demonstration . . . 7

6 Physical layer **7**

6.1 Modulation choices . . . 7

6.2 Waveform generation . . . 8

6.3 Demodulation and timing search . . . 8

6.4 Audible versus near-ultrasonic operation . . . 8

7 Channel metrics and interpretation **9**

8 Web application and data persistence **9**

9 Testing and evaluation **9**

9.1 Reproducible local checks 9

10 Limitations and future work **10**

10.1 Current limitations 10

10.2 Recommended next steps 10

11 Conclusion **11**

A Default configuration quick reference **11**

1 Introduction

Most consumer devices exchange data over radio networks or physical connectors. SonicDrop explores a different channel: a speaker emits a sequence of audio tones and a microphone records them. The design is intended as an educational prototype for short-range, ad-hoc communication, signal-processing experiments, and visualization of link-layer techniques. It does not require the two endpoints to share Wi-Fi or Bluetooth, although the current web application uses a browser and a Django server to prepare and inspect transmissions.

An acoustic room is a difficult communication medium. Reflections can make delayed copies of a tone overlap later symbols; microphone and speaker responses are not flat across frequency; distance and orientation change received level; and unrelated sounds can mask the signal. SonicDrop addresses these issues with multiple tone choices, tapered symbol edges, a trimmed integration region at the receiver, per-carrier energy normalization, frame synchronization, error detection, optional FEC, and repeated packet boundaries.

1.1 Project goals

The implemented project demonstrates the following:

- Encode UTF-8 text and a 16×16 binary image as bits.
- Send bits using audible or near-ultrasonic 2-FSK and 4-FSK.
- Preserve frame boundaries and detect damaged frames using flags, bit stuffing, and CRC-8.
- Correct a single error in an individual Hamming(7,4) codeword when that option is enabled.
- Optionally compress bit strings with a simple, lossless RLE format.
- Generate WAV audio, record microphone audio in the browser, and decode uploaded WAV files.
- Display waveform, frame, BER, SNR, and theoretical channel-capacity information.

1.2 Scope and terminology

The words *sender* and *receiver* refer to the end-to-end roles. The Django application builds a transmission and returns an audio file; the browser can play it or record input. A user can also transfer the WAV file to another device and upload it for decoding. Some interface behavior—notably live packet display and ACK/NACK orchestration—is coordinated between browser contexts, so it should be understood as a demonstration layered around the audio pipeline, not as evidence of a fully autonomous network protocol.

2 System overview

Figure 1 shows the logical path through the project. Encoding proceeds from left to right at the sender; decoding reverses the transformations at the receiver. The channel in between is the physical air path and may add noise, attenuation, timing offset, and multipath effects.

2.1 Repository components

3 End-to-end operation

Figure 2 summarizes a common send/receive session. The sender asks the Django endpoint for an encoded waveform. The result contains WAV bytes and diagnostic metadata; the browser plays it. Independently, a receiver can record audio or decode a WAV file. The decoder tests candidate modem settings, extracts packets, and reconstructs the most recent message identifier.

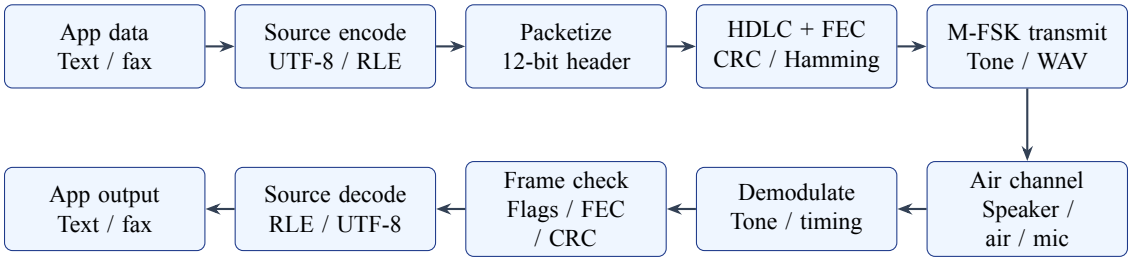


Figure 1: SonicDrop's implemented data path. Optional passkey processing is applied to packet content before link coding; reception filters and isolates audio before frame decoding.

Component	Responsibility
`source_coding.py`	UTF-8/bit conversion, image serialization, RLE, and the optional passkey-based bit transform.
`channel_coding.py`	Packet headers, CRC-8, Hamming(7,4), HDLC flags and bit stuffing, shared-flag streams, and controlled error injection.
`modem_phy.py`	M-FSK waveform generation, WAV serialization, tone-energy demodulation, stream search, and channel-rate calculations.
`views.py`	Django dashboard, transmit/analyze endpoint, WAV decode endpoint, and transmission logging.
`index.html`	Responsive sender, microphone receiver, fax canvas, signal lab, waveform plot, and browser-side coordination.
`models.py`	SQLite-backed transmission history and reported link metrics.

Table 1: Primary implementation modules.

3.1 Transmission endpoint

The `/api/transmit/` endpoint accepts JSON containing the payload type, text or fax bits, carrier band, modulation order, symbol rate, delivery mode, and options for RLE, Hamming coding, noise, error injection, and passkey use. It constructs one or more packet payloads, frames them, synthesizes the complete audio stream, and returns a base64-encoded WAV plus packet timing and diagnostics. Transmission analysis for the signal lab is also performed in the server process.

3.2 Reception endpoint

The `/api/decode-wav/` endpoint accepts a WAV file and receiver settings. It reads mono PCM sample values from the WAV stream, tries a bounded list of modulation configurations, and returns the first configuration that yields valid frames. Frames are CRC-checked, their headers are parsed, optional passkey processing and RLE decoding are applied, and text or fax data are assembled by sequence number. If no valid frames are found, the endpoint returns an error rather than recovered application data.

3.3 Live microphone and browser coordination

The interface uses browser audio APIs to record microphone input and sends WAV recordings to the decode endpoint. A browser-side channel synchronizes settings and demo events across tabs. The sender can also open a short microphone window after playback to look for an ACK/NACK tone. Because live UI packet scheduling and some demonstration controls are coordinated by browser messages, a successful animation or displayed packet should not be confused with a successful independent acoustic decode; the decoded-WAV endpoint is the path that exercises recorded waveform recovery.

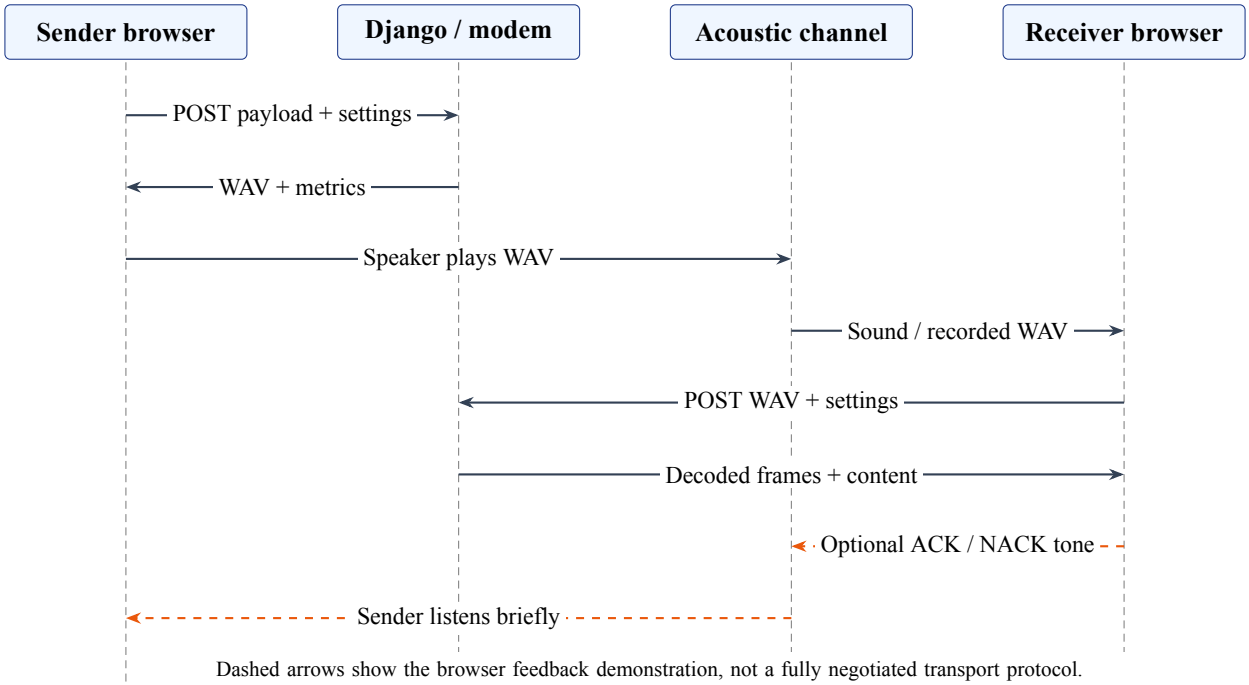


Figure 2: Request, audio, decode, and optional feedback sequence.

4 Source coding and packet format

4.1 Text and image representation

Text is encoded as UTF-8, then represented as eight bits per byte. The decoder truncates incomplete trailing bytes and uses replacement characters for malformed UTF-8; a printable-text check filters likely-noisy results. The 16×16 fax is flattened row by row to 256 binary pixel values. A zero/one pixel is serialized as one bit; the receiver reshapes the recovered sequence into the grid.

4.2 Run-Length Encoding

The RLE format represents each run as one bit for the repeated value followed by a seven-bit unsigned run length. A record is therefore eight bits long; runs are capped at 127 and longer runs are split. For example, a run of five ones is represented by the record `1 0000101`. RLE is optional for text, while fax data is RLE-encoded by the transmit implementation. It is effective for long runs (common in simple binary pictures), but can expand data that alternates frequently because every run consumes eight encoded bits.

4.3 Packet header

Each payload begins with a 12-bit header, followed by application content (or transformed content). Table 2 documents the exact fields.

In word-oriented mode, text is split into words and sent as a sequence of packets; full-message mode groups up to four words in each packet. Fax content is split into fixed-size chunks of compressed bits (32 bits in stream mode and 64 bits in full mode). Packet headers allow the receiver to order data and detect the final packet. The five-bit sequence field limits unambiguous numbering to 32 packet positions; this matters for worst-case fax patterns that compress poorly.

Bit range	Field	Width	Meaning
0–2	Message ID	3	Identifies a message; nonzero values 1–7 are generated.
3	Delivery mode	1	Stream/word-oriented mode flag.
4	Last packet	1	Marks the final packet in the message.
5	Payload type	1	Fax flag; zero denotes text.
6	Encrypted flag	1	Indicates that a passkey transform was applied.
7–11	Sequence number	5	Packet number, represented modulo 32.

Table 2: The packet header packed by the channel-coding module.

4.4 Optional passkey transform and security limits

When a non-empty passkey is supplied, the implementation derives a repeatable bit stream by hashing strings containing a version label, passkey, packet sequence as salt, and counter with SHA-256. It XORs that stream with the payload preceded by an eight-bit SHA-256-derived verification tag. The receiver regenerates the stream, XORs the result, and compares the tag. A packet sequence number is used as the salt so different packets in a message generally have different transform inputs.

Security warning: this is a project demonstration, not a vetted encryption or authentication scheme. The custom construction has no password-hardening KDF, uses a very short (8-bit) verification tag, has no standard AEAD guarantees, and can reuse keystream inputs when the same key and packet sequence recur. It must not be used to protect sensitive information. A production system should use a reviewed authenticated-encryption construction with nonce management and a password-based key derivation function where appropriate.

5 Data-link framing and error control

5.1 CRC-8

The link layer appends an eight-bit CRC remainder computed with generator polynomial

$$G(x) = x^8 + x^2 + x + 1 \qquad \text{(bit pattern } 100000111\text{)}.$$

The CRC is calculated over the packet header and content before FEC. At the receiver, a zero remainder indicates that the received protected data passes the CRC check. CRC detects many accidental corruptions; it is not an error-correction code and should not be treated as cryptographic authentication.

5.2 Hamming(7,4) forward error correction

When enabled, the 12-bit header plus content and CRC are padded to a multiple of four bits and encoded in four-data-bit blocks. Three parity bits are placed in positions 1, 2, and 4, producing seven transmitted bits per four input bits. The decoder calculates a three-bit syndrome; a nonzero syndrome identifies a bit position to flip. This corrects one bit per seven-bit codeword. It does not reliably correct multiple errors in the same codeword, and such errors can be miscorrected; the CRC supplies a second integrity check after decoding.

The code rate is $4/7 \approx 0.571$ before CRC, framing, and bit-stuffing overhead. The extra redundancy increases transmission time but can recover isolated channel bit errors without requesting another copy.

5.3 HDLC-style frame boundaries

The channel coder constructs a frame from a 24-bit alternating preamble, the flag `01111110`, a protected body, and a closing flag. Within the body, bit stuffing inserts a zero after every run of five ones. The decoder removes stuffed zeros before FEC and CRC processing. This reduces the chance that body data will imitate the

flag sequence. For multiple packets, the shared stream includes repeated flags between bodies for additional separation, then a short trailing preamble fragment.

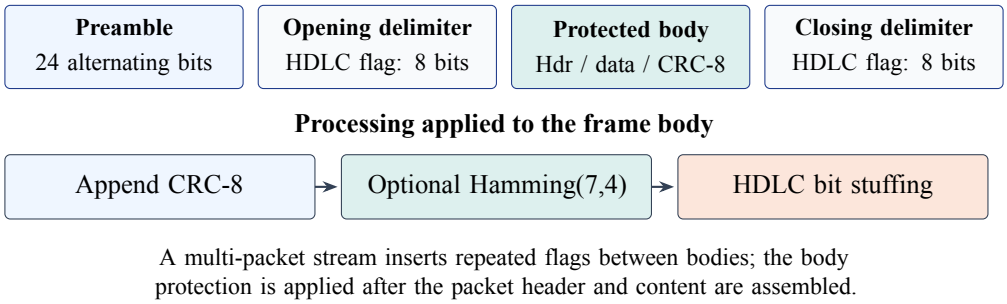


Figure 3: Equal-width frame fields and the order of body protection. Field widths are illustrative; only the stated preamble and flags have fixed bit lengths.

5.4 Packet delivery and retransmission demonstration

The interface labels its feedback feature as Hybrid ARQ. The browser can emit a 6.8 kHz ACK tone or a 3.4 kHz NACK tone and the sender can listen for those frequencies after playback. A forced-NACK option provides a demonstration path that corrupts a selected packet and retries a packet by sequence number; the sender permits at most one automatic retry in that UI flow. The decoder endpoint also returns ACK-related metadata when it has recovered valid frames.

This is not yet a complete selective-repeat ARQ protocol. It does not provide a robust on-air feedback packet with message/sequence identifiers, negotiated timeouts, retry limits across a session, duplicate suppression guarantees, or recovery of missing packets for every receive path. In particular, browser-local simulation, API analysis, and independent WAV reception have different levels of coupling. The report therefore describes ARQ as a prototype feedback demonstration, not as a guarantee of zero data loss.

6 Physical layer

6.1 Modulation choices

The physical layer uses M-ary frequency-shift keying. Each symbol selects one carrier; for $M = 2$ a symbol carries one bit, and for $M = 4$ a symbol carries two bits. The 4-FSK symbol mapping is `00`, `01`, `10`, `11` in the order of the four configured carrier frequencies. If a 4-FSK input contains an odd number of bits, the modulator pads one zero bit.

Mode	M	Carrier frequencies (Hz)	Bandwidth used	Bits/symbol
Audible	2	2600, 3800	2000 Hz	1
Audible	4	2400, 3100, 3800, 4500	2600 Hz	2
Near-ultrasonic	2	14800, 15800	2000 Hz	1
Near-ultrasonic	4	14400, 15000, 15600, 16200	2400 Hz	2

Table 3: Carrier sets and bandwidth values configured in the physical layer.

At sample rate f_s and symbol rate R_s, the implementation uses

$$N_s = \left\lfloor \frac{f_s}{R_s} \right\rfloor$$

samples per symbol. The default sample rate is 44.1 kHz and the default symbol rate is 45 baud. The resulting nominal uncoded bit rate is

$$R_b = R_s \log_2(M).$$

Thus, at 45 baud, 2-FSK carries 45 nominal bits/s and 4-FSK carries 90 nominal bits/s before framing, CRC, FEC, bit stuffing, and optional compression. These are configuration-derived rates, not measured application goodput.

6.2 Waveform generation

The modulator generates a sinusoid at the carrier selected by each symbol. It carries phase forward from symbol to symbol and applies a short raised-cosine taper at each symbol edge to reduce abrupt amplitude transitions. The resulting float waveform is scaled and converted to mono, signed 16-bit PCM WAV. For 4-FSK, an odd trailing bit is padded as noted above.

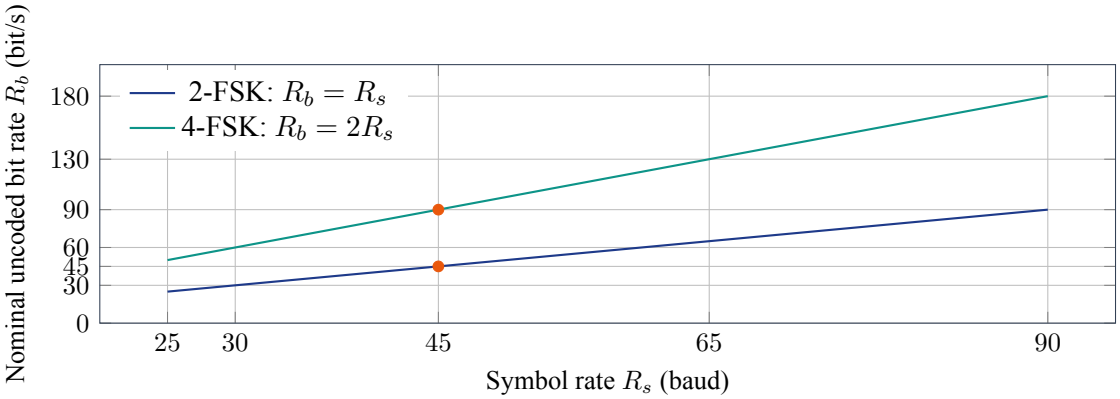


Figure 4: Nominal modulation bit rate versus symbol rate, calculated from $R_b = R_s \log_2(M)$. Markers show the project's default 45-baud setting. The plot excludes protocol overhead and does not represent measured throughput.

6.3 Demodulation and timing search

For each symbol, the receiver discards an edge guard region (approximately 14% of the symbol at each end) and correlates the remaining samples against in-phase and quadrature templates for each candidate carrier. The squared correlation magnitudes estimate tone energy. When enough symbols are present, each carrier's energy is normalized by a high percentile of its observed power to partially compensate for unequal speaker/microphone frequency response. The highest normalized-energy carrier is selected; low-energy symbols are squelched.

For WAV reception, the implementation removes the recording mean, applies an FFT band-pass mask around the configured carrier range, normalizes gain, and isolates an active burst using tone dominance. It then tries up to 20 offsets within one symbol interval. At each offset, the resulting bits are scanned for frame flags and checked with FEC and CRC. This search helps tolerate unknown start alignment, but is a bounded timing search rather than a full timing-recovery loop or channel equalizer.

6.4 Audible versus near-ultrasonic operation

The near-ultrasonic carrier set aims to reduce audible tones while remaining within the nominal Nyquist limit of a 44.1 kHz sample rate. Real hardware may attenuate these frequencies strongly or reject them through built-in audio processing; some microphones, speakers, and codecs do not preserve them adequately. The audible setting is generally easier to observe and debug. Neither carrier mode should be assumed to work across all devices without measurement.

7 Channel metrics and interpretation

The physical layer reports an estimated SNR in decibels, a nominal bit rate, a Nyquist rate, and a Shannon capacity estimate. In the code these estimates use the detected tone and residual competing-tone powers:

$$\mathrm{SNR}_{dB} = 10 \log_{10}\left(\frac{P_s}{P_n}\right),$$

where P_s is estimated average winning-carrier power and P_n is average power in the other carrier hypotheses. The reported theoretical limits are

$$R_{\text{Nyquist}} = 2B \log_2(M), \qquad C_{\text{Shannon}} = B \log_2(1 + \mathrm{SNR}_{linear}),$$

with configured bandwidth B. These are reference calculations under idealized assumptions; the acoustic channel is not guaranteed to satisfy those assumptions, and an estimated SNR is not a lab measurement of end-to-end bit error probability.

The transmit-and-analyze endpoint returns a clean synthesized WAV, then applies Gaussian noise and optional artificial bit flips to a separate first-frame copy for analysis. Its BER is computed by comparing that demodulated copy with the generated frame. This metric is useful for controlled tests but is not a field BER measurement. In the WAV receive path, the reported BER-like value is derived from corrected bits and a displayed frame length; it should not be interpreted as a complete statistical estimate over many independent packets.

8 Web application and data persistence

Django exposes three routes: the dashboard at `/`, transmission analysis at `/api/transmit/`, and WAV decoding at `/api/decode-wav/`. NumPy implements waveform and spectral operations. The dashboard provides text and fax input, controls for carrier/modulation and link options, a recording receiver, WAV upload, a frame bit inspector, metrics cards, and a waveform plot. The fax grid is 16×16 pixels.

Each transmit-and-analyze request creates a `TransmissionLog` record in SQLite with the chosen band, symbol-rate-related field, coding options, bit lengths, injected/corrected error counts, BER, SNR, theoretical rates, CRC result, and recovered/display label. The dashboard shows recent entries. This history supports experiments but is not a production telemetry or account system.

9 Testing and evaluation

The repository includes an engine demonstration that:

1. Converts a UTF-8 test sentence into bits, frames it with CRC-8 and Hamming coding, injects artificial errors, and prints decoder correction and CRC results.
2. Encodes and decodes an example binary image with RLE and verifies that the output bits match the input bits.

The README documents Django's `manage.py check` and an optional speaker/microphone demo. These checks are not a cross-device benchmark; no range, error-rate, security, or hardware-compatibility guarantees are claimed.

9.1 Reproducible local checks

With project dependencies installed, the documented checks are:

```
python manage.py check
python transceiver/engine/test_engine.py
```

For an end-to-end acoustic evaluation, record the emitted WAV with a second device at documented distance, orientation, volume, and environment settings. Repeat each condition over many packets, report both packet success rate and goodput, and retain the original recordings so decoding results can be reproduced.

10 Limitations and future work

10.1 Current limitations

- **Prototype cryptography:** the SHA-256/XOR transform and short tag are not safe for confidential data; use a standard authenticated-encryption library for any real security requirement.
- **Incomplete ARQ:** feedback and retry behavior are primarily a browser demonstration. It lacks a complete on-air protocol for identifiers, timeout policy, loss recovery, and duplicate handling.
- **Packet-size and sequence bounds:** the header has only five sequence bits. Highly alternating fax pixels can produce enough RLE records to exceed 32 packet positions.
- **Timing and channel model:** the receiver uses a finite phase search, energy detector, and simple spectral compensation. It does not estimate a full impulse response or continuously track clock drift.
- **Rate adaptation:** the code produces a recommended profile from SNR/BER thresholds; this recommendation is diagnostic and is not, by itself, a negotiated automatic change to an ongoing link.
- **Limited measurements:** synthetic error injection and the included engine demonstration do not establish acoustic range, robustness across rooms, or real-world throughput.
- **History field naming:** the transmit view currently stores effective bit rate in the database field named `baud_rate`; historical values should not be read as symbol-rate measurements.
- **Input validation and deployment:** this is a local development application. Before public deployment, enforce request-size limits, validate all options, review CSRF/API policy, configure production secrets and allowed hosts, and use an appropriate server configuration.

10.2 Recommended next steps

1. Add automated unit tests for CRC vectors, framing/unstuffing, single- and multi-bit Hamming cases, RLE edge cases, packet boundaries, encryption failure behavior, and modem round trips.
2. Add a versioned frame format with payload length, robust message identifiers, and sequence numbers large enough for the supported maximum message size.
3. Replace the custom passkey transform with a standard AEAD scheme and define a secure key derivation and nonce lifecycle.
4. Specify an actual feedback frame or robust coded tone sequence, including packet identity, timeout, retry, and duplicate rules; test it on independent devices.
5. Measure recorded audio across devices and environments. Publish sample rate, distance, volume, carrier set, baud rate, packet size, trials, error counts, and confidence intervals.
6. Improve synchronization and channel estimation, and evaluate interleaving or stronger codes for burst noise and reverberation.
7. Separate simulator-generated metrics from measurements on recordings in the UI and stored history.

11 Conclusion

SonicDrop is an end-to-end educational acoustic modem: it turns application data into framed bit streams, maps those streams to audio tones, and provides a software receiver for recovering content from recordings. Its layered implementation makes source coding, packetization, CRC, optional FEC, M-FSK, signal metrics, and web presentation visible and testable. The included diagrams and rate plot describe the implemented algorithms and theoretical calculations rather than claiming a benchmark result. The project is a useful foundation for further experimentation, with cryptography, protocol completeness, acoustic evaluation, and stronger synchronization identified as the most important next steps.

A Default configuration quick reference

Parameter	**Default / behavior**
Sample rate	44,100 samples/s; mono 16-bit PCM WAV output.
Symbol rate	45 baud in the API defaults and initial interface control; the UI slider spans 25–90 baud in 5-baud steps.
Modulation	4-FSK by default (two bits per symbol); 2-FSK is also selectable.
Carrier band	Audible by default; near-ultrasonic is selectable.
Hamming	Enabled by default; may be disabled.
Text RLE	Disabled by default; may be enabled. Fax content is always RLE encoded by the transmit pipeline.
Noise/error controls	The analysis endpoint supports configurable Gaussian noise and optional artificial single, multiple, or burst bit flips.
WAV receive search	Tries requested settings and several fallback configurations, then returns the first set of valid frames found.

Table 4: Defaults and observable behavior in the current implementation.